Laboratório Funcional · Aula 12

Joao Gabriel Caldeira Milare · RA 241136722 · Turma Esoft 6 A
Gerado em 16/10/2026, 20:55:58 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora(): É impura porque o seu resultado depende de um estado externo (o relógio do sistema), devolvendo valores diferentes a cada chamada. poe(lista, x): É impura porque tem o efeito colateral de modificar o estado da lista original fornecida como parâmetro, utilizando o método

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2
4: 14
7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido



1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") dá resultados diferentes porque ela é impura. Ela depende e altera uma variável global (contador) que está fora dela. Isso é ruim para os testes porque a função não funciona mais de forma isolada; o resultado dela passa a depender do estado atual do programa inteiro. Para o teste dar certo, você sempre teria que preparar ou resetar essa variável global antes de chamar a função.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

A diferença acontece porque a ordem das operações matemáticas altera o resultado, e a composição resolve de dentro para fora. No compor(f, g)(4), o código roda a g(4) primeiro (3 * 4 = 12) e joga esse 12 na f (12 + 2 = 14).No compor(g, f)(4), ele inverte: roda a f(4) primeiro (4 + 2 = 6) e joga esse 6 na g (3 * 6 = 18).

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

O sorted(notas) pega a sua lista, cria uma cópia nova em folha, ordena e devolve, deixando a original intacta. O notas.sort() vai direto na lista original e altera a ordem dos itens na memória. A abordagem funcional é o sorted, já que a regra de ouro do paradigma funcional é manter a imutabilidade dos dados e evitar efeitos colaterais (como modificar a lista que a função recebeu).

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

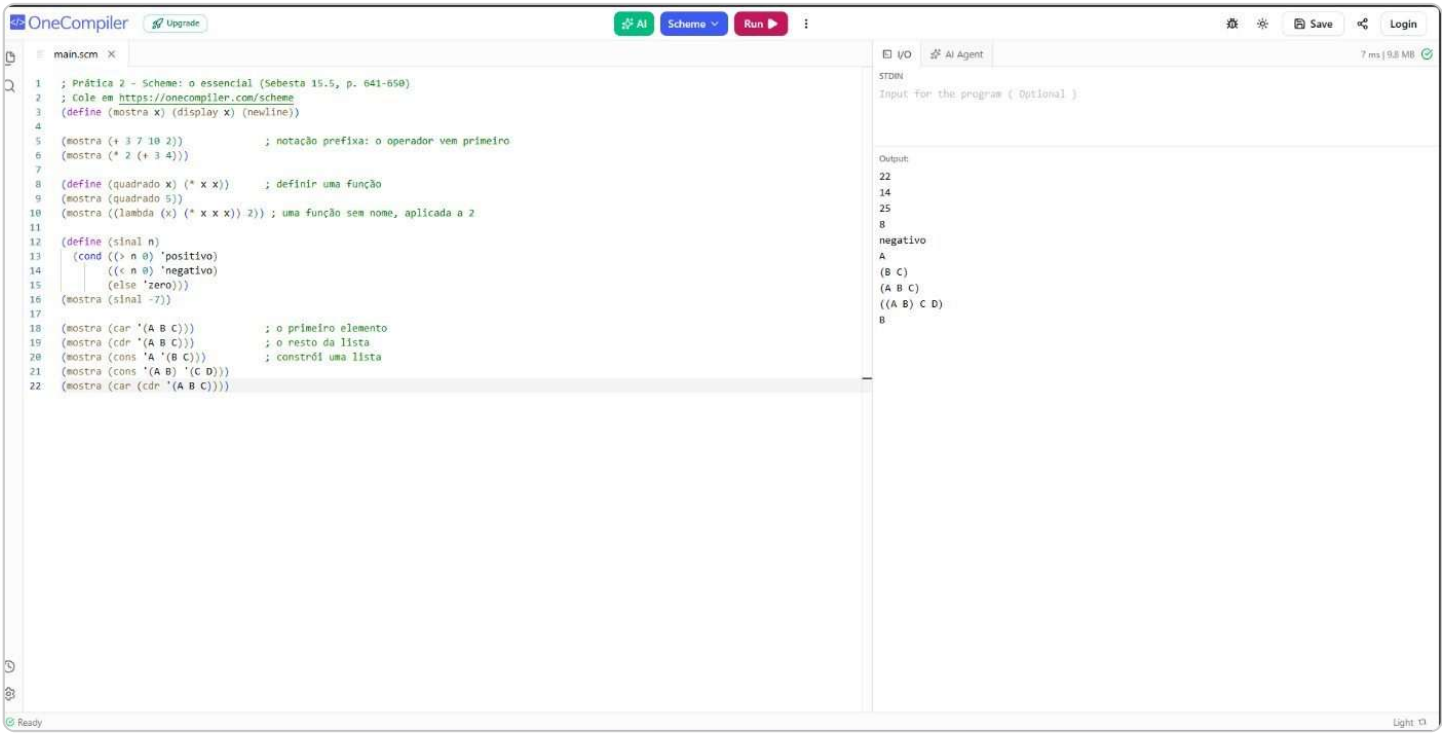
B C

Previsão: (cons '(A B) '(C D))

A B C D

Saídas (original e sem o apóstrofo)

não respondido




```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))            ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))    ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car '(A B C)))          ; o primeiro elemento
(mostra (cdr '(A B C)))          ; o resto da lista
(mostra (cons 'A '(B C)))        ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') é uma abreviação da função QUOTE. Ele serve para avisar ao Scheme que aquilo é um dado (uma lista de elementos ou um símbolo) e não um código para ser executado. Quando você tira o apóstrofo e deixa apenas (A B C), a regra de avaliação do Scheme entra em ação: ele acha que o primeiro elemento (A) é uma função e tenta chamá-la passando B e C como parâmetros. Como A não é uma função que existe, o programa dá um erro dizendo que a variável não está definida.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
Explicação passo a passo para entender:
O primeiro (cdr '(A B C)) devolve a lista sem o "A", ou seja: (B C).  O segundo cdr é
aplicado em cima desse resultado, devolvendo a lista sem o "B", restando apenas: (C).  Por
fim, o car pega o primeiro (e único) elemento que sobrou nessa lista final, devolvendo o C
sozinho.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 5: comum em 10 passos, até 4 multiplicações esperando (expressão de até 36 caracteres); em cauda em 7 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

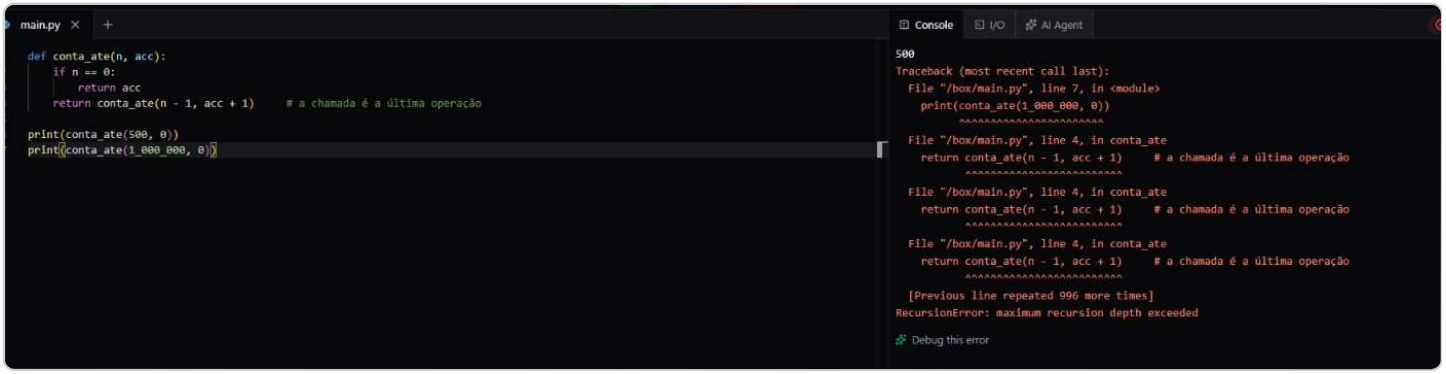
No Scheme vai funcionar, porque conta-ate é recursiva em cauda , então o Scheme reaproveita o registro de ativação e não empilha um milhão de chamadas. Sendo assim, ele deve imprimir um milhao. No Python não vai funcionar, porque ele não otimiza a recursão em cauda e cada chamada empilha um registro novo. Como o Python tem um limite de chamadas empilhadas, ele vai acabar causando u erro de recursão antes de chegar no um milhão, mesmo a função sendo igual à do Scheme.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, o que cresce é a pilha de chamadas. Em, o 4 multiplica o resultado de, que espera , e assim por diante, então cada chamada fica parada com a multiplicação pendente. Só quando chega no 1 as contas voltam e a pilha esvazia. Na recursão em cauda isso não acontece, porque o resultado parcial vai no parâmetro parcial e a chamada recursiva é a última coisa que a função faz. Sendo assim, nada fica esperando a volta e a pilha não precisa crescer.

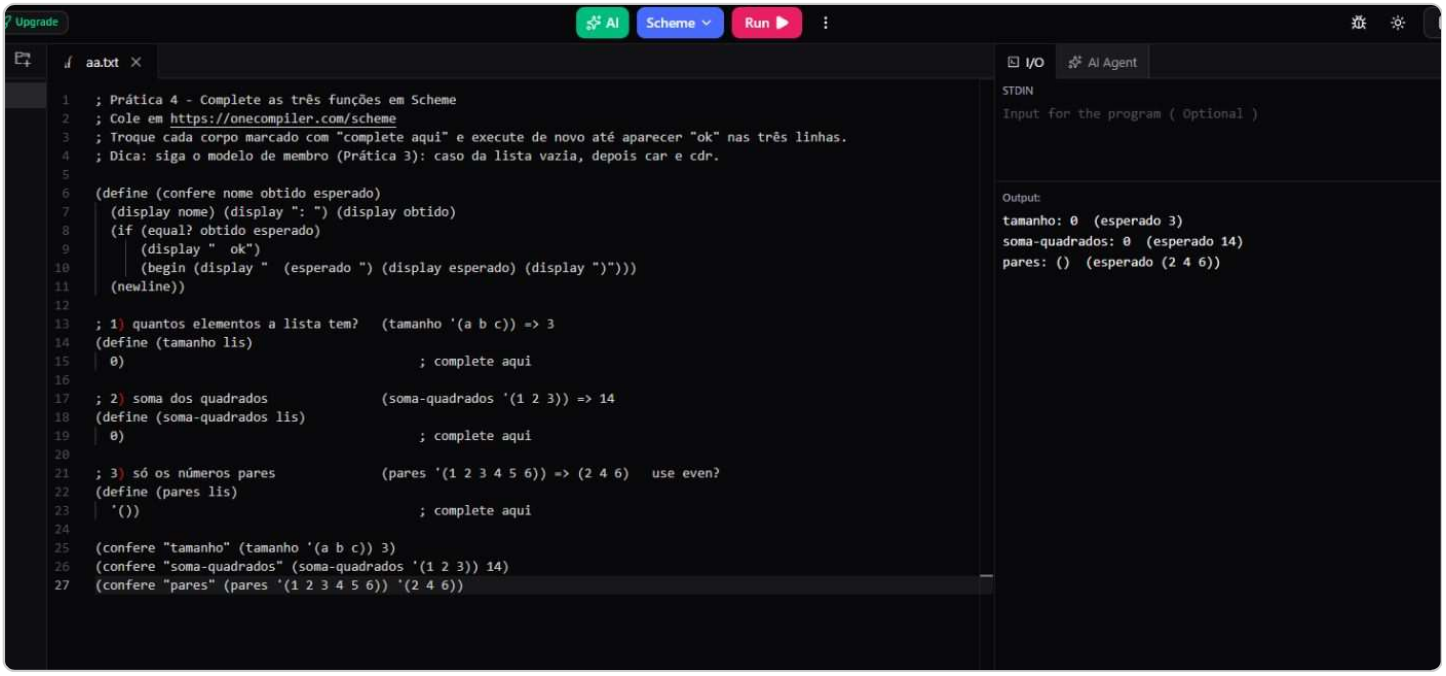
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

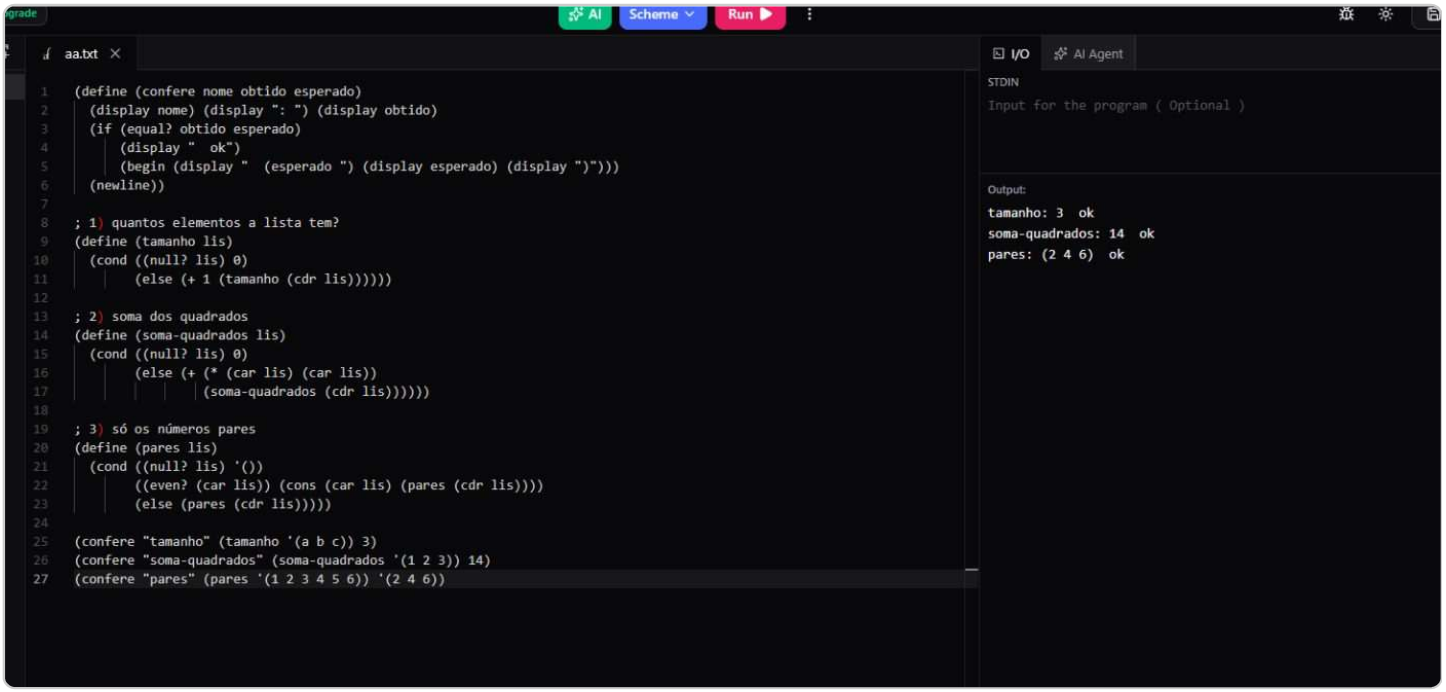
O Scheme fez o milhão de chamadas porque faz a otimização da chamada em cauda: como conta-ate não deixa nada pendente, ele aproveita o mesmo registro de ativação, e a recursão ocupa espaço constante, igual a um laço. O Python não faz essa otimização, então cada chamada empilha um registro novo, mesmo a função sendo em cauda. Como ele também limita a profundidade da pilha, o conta_ate(500, 0) funciona e o de 1milhão e estoura com o RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido





4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil para fazer foi a pares, porque ela tem três casos e eu precisei lembrar de usar o cons para montar a lista nova. Em todas as funções eu usei o null primeiro, para ver se a lista acabou e devolver a resposta direta . Sendo assim, o car pega o primeiro elemento: no soma-quadrados ele é elevado ao quadrado, e no pares ele é testado com o even e colocado na frente com o cons. Então o cdr entra na chamada recursiva, para continuar com o resto da lista até ela ficar vazia. No tamanho o car nem é usado, só o cdr, porque o que importa é somar 1 a cada chamada.

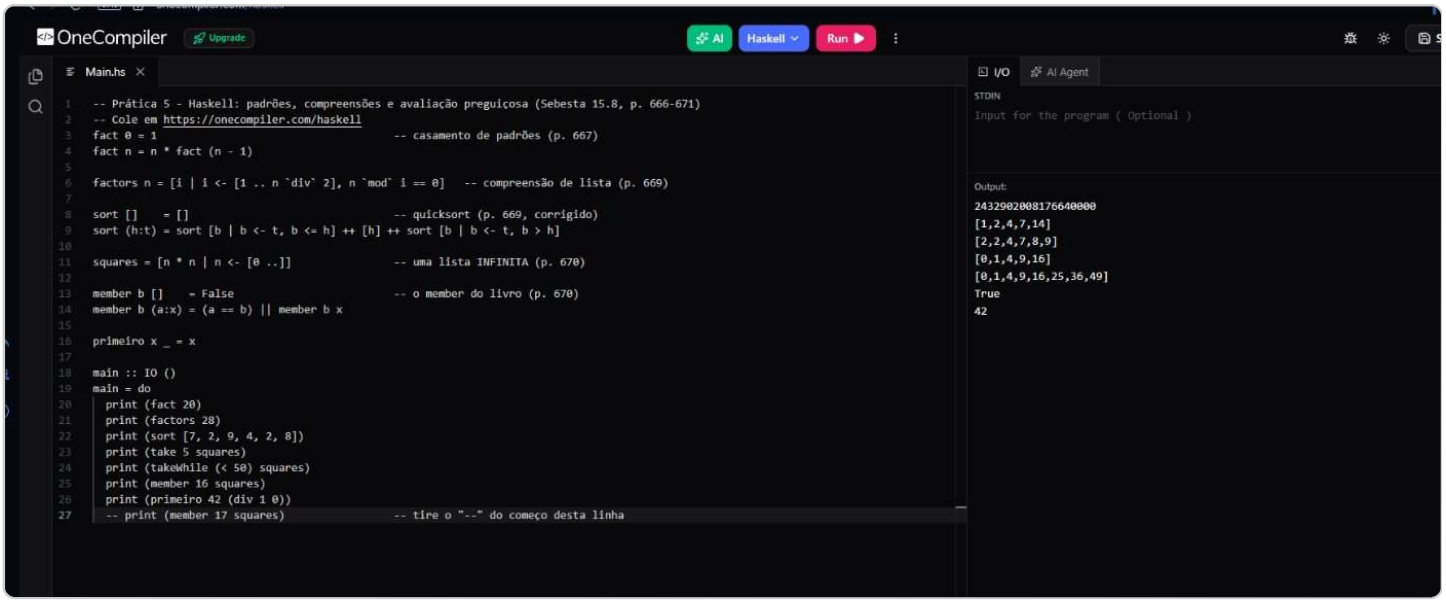
Prática 5: Haskell: padrões, compreensões e preguiça

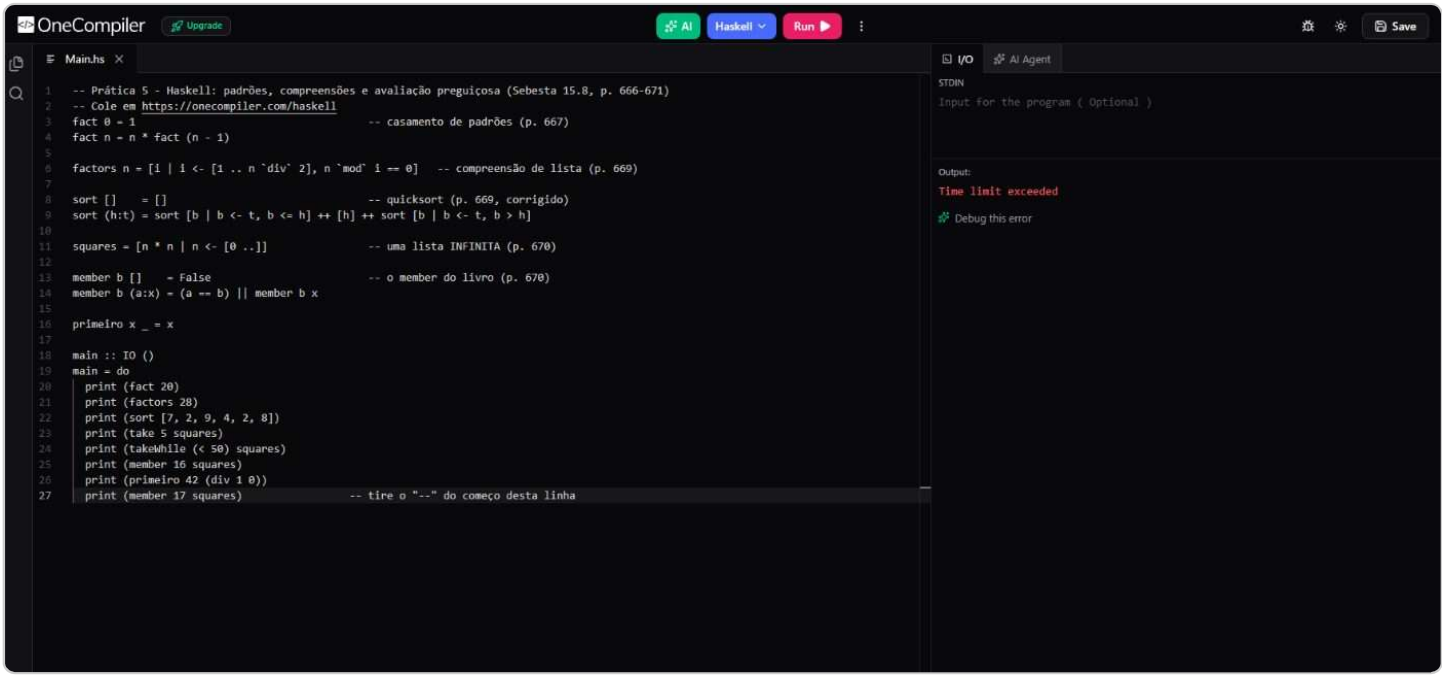
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

Eu acho que take 5 squares vai imprimir 0,1,4,9,16. A lista squares é infinita, mas o Haskell só calcula quando precisa do valor, e o take 5 pede só os cinco primeiros. Sendo assim, ele calcula 0, 1, 4, 9 e 16 e para, sem tentar gerar o resto da lista. Já o primeiro 42 eu acho que vai imprimir 42 sem dar erro. A função primeiro ignora o segundo parâmetro o "_", e como o Haskell não calcula o que a função não usa, o div 1 0 nunca chega a ser executado. Então não tem divisão por zero.

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa basicamente só calcula um valor quando precisa dele. Sendo assim, o take 5 squares calcula apenas os cinco primeiros elementos e para, sem gerar o resto da lista infinita. O primeiro 42 (div 1 0) não deu erro porque a função ignora o segundo parâmetro o _Então o div 1 0 nunca foi calculado e o resultado foi só 42.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 17 squares fica meio q rodando sem parar porque o 17 não é quadrado perfeito, então nunca aparece na lista. Como a lista é infinita, a busca não chega ao fim. O member 16 squares terminou porque o 16 está na lista (4 · 4) e o a == b deu verdadeiro. Para corrigir, dá para aproveitar que squares está em ordem crescente. Quando o elemento a já for maior que o b procurado e acaba desistindo e retornando false.

member b [] = False
member b (a:x) = (a == b) || (a < b && member b x)

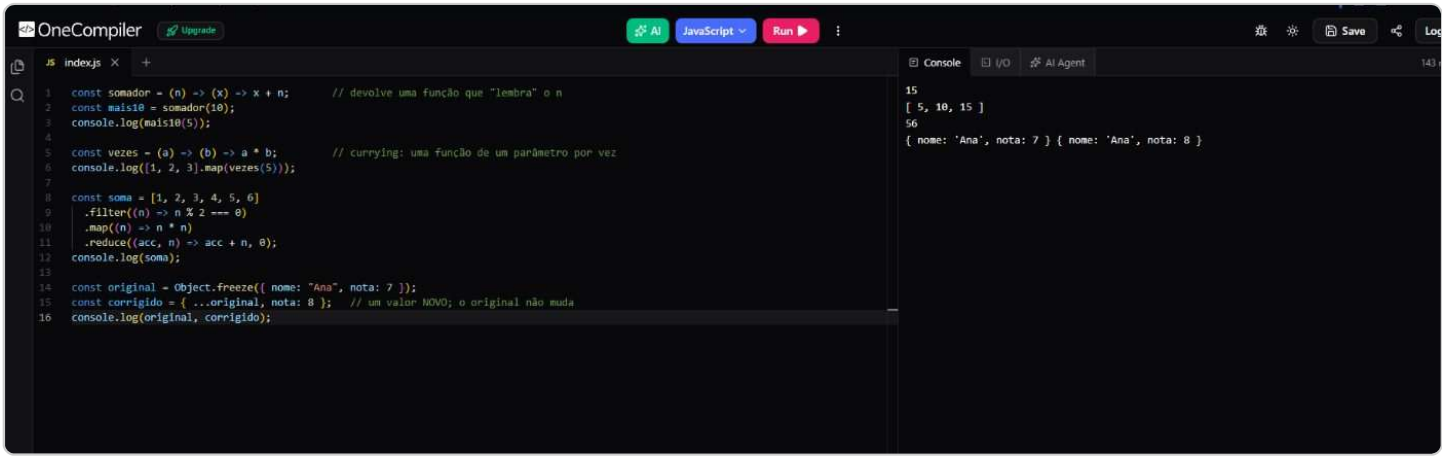
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) deve imprimir 15, e 1, 2, 3.map(vezes(5)) deve imprimir 5, 10, 15.

Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida por somador(10) é um fechamento: ela guarda uma ligação com o ambiente onde foi criada. Sendo assim, mesmo depois de o somador terminar, o n que vale 10 continua existindo enquanto a função existir. Então, quando chamamos mais10(5), ela ainda enxerga o 10 e devolve 5 + 10 = 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Deram resultados diferentes porque a ordem de aplicação é diferente. O f.compose(g) aplica primeiro o g e depois o f, ou seja, f(g(4)) = 3 · 4 + 2 = 14. O f.andThen(g) aplica primeiro o f e depois o g, ou seja, g(f(4)) = 3 · (4 + 2) = 18. O que corresponde ao compor(f, g) da prática 1 é o compose, porque nos dois casos o g é aplicado primeiro e o resultado é 14.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();
```

```
System.out.println(soma);
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const aprovados = alunos
    .filter((aluno) ⇒ aluno.nota ⩾ 6)
    .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244

ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional fica mais simples de entender em alguns casos, porque dá pra ver direto o que está sendo feito com os dados. Já com o laço, acho que fica mais fácil acompanhar cada passo da execução e entender o que está acontecendo.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell fica mais simples e direta de entender, porque dá pra fazer várias operações sem precisar escrever tantos comandos. Eu usaria essa abordagem em situações em que a legibilidade e a manutenção do código fossem mais importantes, mas evitaria em sistemas em que o desempenho fosse muito importante.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me chamou a atenção foi ver que dá pra filtrar, transformar e somar os valores sem precisar usar laços e variáveis acumuladoras, deixando o código mais direto.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está na parte b <- h, porque o h é um único valor e não uma lista. O correto seria usar b <= h para separar os valores menores ou iguais ao pivô.